

Machine Learning and Data Mining

Classification - Part III

Claudio Sartori

DISI

Department of Computer Science and Engineering – University of Bologna, Italy

claudio.sartori@unibo.it

1	Statistical modeling – Naive Bayes Classifier	2
•	A fictitious example	4
•	The Bayes method	6
•	Missing values	10
•	Numeric values	11
•	Summary	16
2	Linear Classification with the Perceptron	17
3	Support Vector Machines	24
4	Neural Networks	42
5	Instance Learning - K Nearest Neighbours Classifier	66
6	Loss function	68
7	From a binary classifier to multi-class classification	70
8	Ensemble methods	75
9	Forest of randomised trees	85
10	Boosting	89

Naive Bayes Classifier ²

Main issues

- Based on statistics
 - In particular, on the Bayes' theorem
- Consider the contribution of all the attributes
- Assume that each attribute is **independent** from the others, *given the class*¹
 - This is a very strong assumption, rarely verified, but, nevertheless, the method works!
- Estimate the probabilities with the frequencies, as usual

1 This means $\Pr(d_1 = v_1, d_2 = v_2 \mid c = c_x) = \Pr(d_1 = v_1 \mid c = c_x) \cdot \Pr(d_2 = v_2 \mid c = c_x)$

2 Description based on [Witten et al.(2011)Witten, Frank, and Hall]

The weather/play data

Numbers of cases and fractions of the Weather/Play dataset

Outlook			Temperature			Humidity			Windy			Play	
	yes	no		yes	no		yes	no		yes	no	yes	no
sunny	2	3	hot	2	2	high	3	4	false	6	2	9	5
overcast	4	0	mild	4	2	normal	6	1	true	3	3		
rainy	3	2	cool	3	1								
sunny	2/9	3/5	hot	2/9	2/5	high	3/9	4/5	false	6/9	2/5	9/14	5/14
overcast	4/9	0/5	mild	4/9	2/5	normal	6/9	1/5	true	3/9	3/5		
rainy	3/9	2/5	cool	3/9	1/5								

A new sample needs classification

Outlook: sunny, Temperature: cool, Humidity: high, Windy: true, Play: ?

- Treat the five features and the overall likelihood that *play* is *yes* or *no* as equally important
 - they are independent pieces of evidence, the overall likelihood is obtained by multiplying the probabilities (i.e. the frequencies)

$$\text{likelihood of yes} = 2/9 \times 3/9 \times 3/9 \times 3/9 \times 9/14 = 0.0053$$

$$\text{likelihood of no} = 3/5 \times 1/5 \times 4/5 \times 3/5 \times 5/14 = 0.0206$$

- Normalize to 1

$$\Pr(\text{yes}) = \frac{0.0053}{0.0053 + 0.0206} = 20.5\%$$

$$\Pr(\text{no}) = \frac{0.0206}{0.0053 + 0.0206} = 79.5\%$$

- *no* is more likely than *yes*, about four times

The Bayes' theorem

Given a hypothesis H and an evidence E that bears on that hypothesis

$$\Pr(H | E) = \frac{\Pr(E | H) \Pr(H)}{\Pr(E)}$$

- The hypothesis is the class, say c , the evidence is the tuple of values of the element to be classified
- We can split the evidence into pieces, one per attribute, and, if the attributes are **independent** inside each class,

Assume $E = \{E_1, E_2, E_3, E_4\}$

$$\Pr(c | E) = \frac{\Pr(E_1 | c) \times \Pr(E_2 | c) \times \Pr(E_3 | c) \times \Pr(E_4 | c) \times \Pr(c)}{\Pr(E)}$$

$\Pr(E | c) \leftarrow$

The Naive Bayes method

- Compute the conditional probabilities from examples
- Apply the theorem
- The denominator is the same for all the classes, and is eliminated by the normalization step
- It is called *naive* since the assumption of independence between attributes is quite simplistic
 - Nevertheless it works quite well in many cases

Problem

What if value v of attribute d never appears in the elements of class c ?

- In this case $\Pr(d = v | c) = 0$ **The whole numerator of the bayes formula would become 0**
- This makes the probability of the class for that evidence drop to zero
- In practice, this case is quite common, in particular in a domain with many attributes and many distinct values
- An alternative solution is needed  **Laplace smoothing**

Values not represented in a class – Laplace smoothing

α – Smoothing parameter, typical value is 1

$af_{d=v_i,c}$ – **Absolute** frequency of value v_i in attribute d over class c

V – number of distinct values in attribute d over the dataset

af_c – **Absolute** frequency of class c in the dataset

$$\text{Smoothed frequency } sf_{d=v_i,c} = \frac{af_{d=v_i,c} + \alpha}{af_c + \alpha V}$$

If alpha $\rightarrow$ inf :
 $(x + \text{inf}) / (y + V * \text{inf}) =$
 $(x/\text{inf} + 1) / (y/\text{inf} + V) =$
 $= 1/V$
 (average probability of
 the distinct values)

- With $\alpha = 0$ we obtain the standard, unsmoothed formula
- **Higher** values of α give more importance to the prior probabilities for the values of d w.r.t. the evidence given by the examples

Missing values

They do not affect the model, it is not necessary to discard an instance with missing value(s)

- **Test** instance:
 - The calculation of the likelihood simply omits this attribute
 - The likelihood will be higher for all the classes, but this is compensated by the normalization
- **Train** instance:
 - The record is simply not included in the frequency counts for that attribute
 - The descriptive statistics are based on the number of values that occur, rather than on the number of instances

Numeric values

- The method based on frequencies is inapplicable
- Additional assumption: the values have a *Gaussian* distribution
- Instead of the fractions of counts, we compute, from the examples the mean μ and the variance σ of the values of each numeric attribute inside each class
- For a given attribute and a given class, the distribution is supposed to be

$$f(x) = \frac{1}{\sqrt{2\pi\sigma}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

The weather/play data with numeric values

Numbers of cases, fractions and descriptive statistics of the Weather/Play dataset

Outlook			Temperature		Humidity		Windy			Play			
	yes	no	yes	no	yes	no	yes	no	yes	no			
sunny	2	3	83	85	86	85	false	6	2	9	5		
overcast	4	0	70	80	96	90	true	3	3				
rainy	3	2	68	65	80	70							
			64	72	65	95							
			69	71	70	91							
			75		80								
			75		70								
			72		90								
			81		75								
sunny	2/9	3/5	mean	73	74.6	mean	79.1	86.2	false	6/9	2/5	9/14	5/14
overcast	4/9	0/5	stdev	6.2	7.9	stdev	10.2	9.7	true	3/9	3/5		
rainy	3/9	2/5											

Using numeric values in Naive Bayes

- We are considering a *yes* outcome when the temperature is 66
- Plug the the value under consideration, the mean and the stdev in the gaussian probability density formula

$$f(\text{temperature} = 66|\text{yes}) = \frac{1}{\sqrt{2\pi} \cdot 6.2} e^{-\frac{(66-73)^2}{2 \cdot 6.2^2}} = 0.0340$$
$$f(\text{humidity} = 90|\text{yes}) = 0.0221$$

Using numeric values in Naive Bayes

Probability and probability density are closely related, but are not the same thing

- On a continuous domain, the probability of a variable assuming *exactly* a single real value is zero
- A value of the density function is the probability that the variable lies in a small interval around that value
- The value we use are, of course, rounded at some precision factor
- That precision factor is the same for all the classes, then we can disregard it
- If numeric values are missing, mean and standard deviation are based only on the values that are present

Classification of a sample with numeric values

Outlook: sunny, Temperature: 66, Humidity: 90, Windy: true, Play: ?

$$\text{likelihood of } \textit{yes} = 2/9 \times 0.0340 \times 0.0221 \times 3/9 \times 9/14 = 0.000036$$

$$\text{likelihood of } \textit{no} = 3/5 \times 0.0221 \times 0.0381 \times 3/5 \times 5/14 = 0.000108$$

$$\Pr(\textit{yes}) = \frac{0.000036}{0.000036 + 0.000108} = 25.0\%$$

$$\Pr(\textit{no}) = \frac{0.000108}{0.000036 + 0.000108} = 75.0\%$$

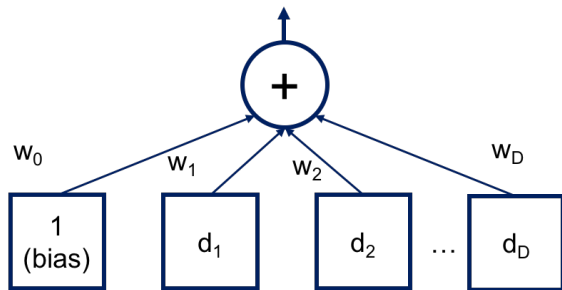
Summary

- Clear semantics for learning probabilistic knowledge
- Excellent results in many cases
- Dramatic degradation when the simplistic conditions are not met
 - Violation of *independence* – for instance, if an attribute is simply a copy of another (or a linear transformation), the weight of that particular feature is enforced (something like squaring the probability)
 - Violation of *gaussian* distribution – use the standard probability estimation for the appropriate distribution, if known, or use estimation procedures, such as *Kernel Density Estimation*

1	Statistical modeling – Naive Bayes Classifier	2
2	Linear Classification with the Perceptron	17
•	Training the perceptron	21
•	Convergence	22
3	Support Vector Machines	24
4	Neural Networks	42
5	Instance Learning - K Nearest Neighbours Classifier	66
6	Loss function	68
7	From a binary classifier to multi-class classification	70
8	Ensemble methods	75
9	Forest of randomised trees	85
10	Boosting	89
11	Summary	95

The linear perceptron ³

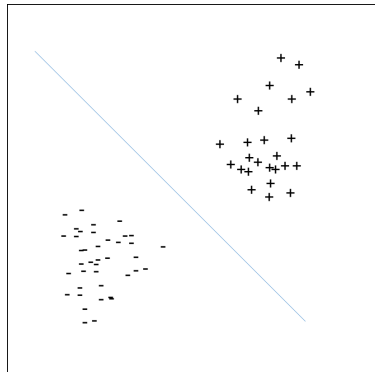
- Often called also **artificial neuron**
- In practice, a linear combination of weighted inputs



3 Description based on [Witten et al.(2011)Witten, Frank, and Hall]

Separate examples of two classes

- For a dataset with numeric attributes
- Learn a *hyperplane* such that all the positives lay on one side and all the negatives on the other



The hyperplane

- The hyperplane is described by a set of weights $w_0, \dots, w_D$ in a linear equation on the data attributes $x_0, \dots, x_D$
 - the fictitious attribute $x_0 = 1$ is added to allow a hyperplane that does not pass through the origin
- There are either **none** or **infinite** such hyperplanes

$$w_0 * x_0 + w_1 * x_1 + \dots + w_D * x_D \quad \begin{cases} > 0 \Rightarrow \text{positive} \\ < 0 \Rightarrow \text{negative} \end{cases}$$

Learning the hyperplane

set all weights to zero

while there are examples incorrectly classified **do**

for each training instance x **do**

if x is incorrectly classified **then**

if class of x is positive **then** (Change the coefficients)

 add the x data vector to the vector of weights

else

 subtract the x data vector from the vector of weights

Linear perceptron convergence

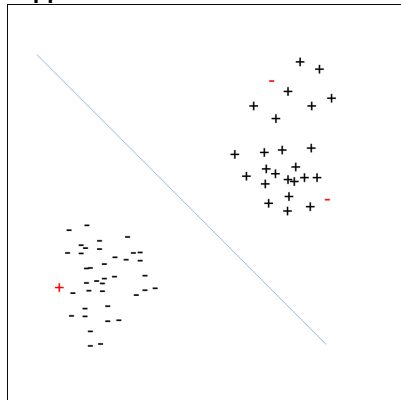
- Each change of weights moves the hyperplane towards the misclassified instance, consider the equation after the weight change for a positive instance x which was classified as negative
$$(w_0 + x_0) * x_0 + (w_1 + x_1) * x_1 + \dots + (w_D + x_D) * x_D$$
- The result of the equation is increased by a **positive** amount
$$x_0^2 + \dots + x_D^2$$
- Therefore the result will be **less negative** or, possibly, even **positive**
- Analogously for a negative instance which was classified as positive

Linear perceptron algorithm termination

And maximize the distance

One idea for set the hyperplane is taking the two nearest points of different classes and place a line in the middle (maximum margin hyperplane) which is used in support vector machines

- The corrections are incremental and can interfere with previous updates
- The algorithm converges if the dataset is linearly separable, otherwise it does not terminate
- For practical applicability it is necessary to set an upper bound to the iterations



1	Statistical modeling – Naive Bayes Classifier	2
2	Linear Classification with the Perceptron	17
3	Support Vector Machines	24
•	• The Maximum Margin hyperplane	28
•	• Non-linear class boundaries	34
4	Neural Networks	42
5	Instance Learning - K Nearest Neighbours Classifier	66
6	Loss function	68
7	From a binary classifier to multi-class classification	70
8	Ensemble methods	75
9	Forest of randomised trees	85
10	Boosting	89
11	Summary	95

Support Vector Machines (SVM) for binary classification I

Limitations of the linear models

Description of SVM based on [Witten et al.(2011)Witten, Frank, and Hall]

- What can we do if the data are not *linearly separable*?
 - This means that the boundary between classes is some hyper-surface more complex than a hyperplane
- One possibility would be to give up the linearity, e.g.:

$$w_1 * x_1^3 + w_2 * x_1^2 * x_2 + w_3 * x_1 * x_2^2 + w_4 * x_3^3$$

Support Vector Machines (SVM) for binary classification II

Limitations of the linear models

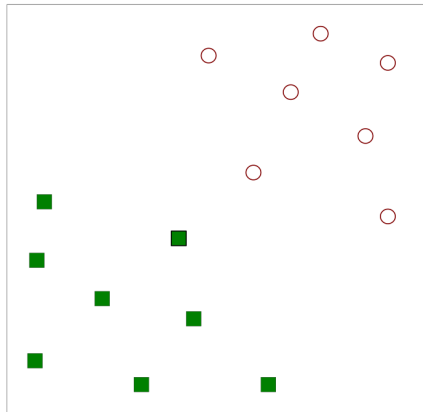
Description of SVM based on [Witten et al.(2011)Witten, Frank, and Hall]

- The method would become soon intractable for any reasonable number of variables
 - with 10 variables and limiting to factors with maximum order 5 we would need something like 2000 coefficient
- The method would be extremely prone to overfitting, if the number of parameters approaches the number of examples

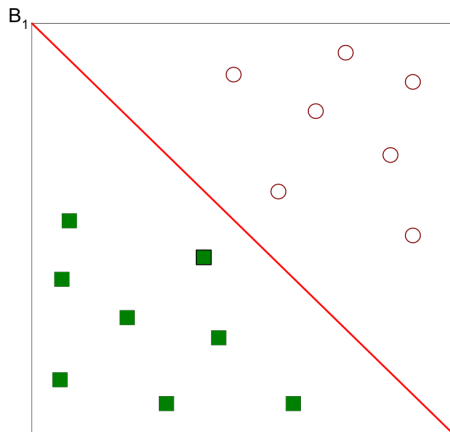
Key ideas

- Computational learning theory
- New efficient separability of non-linear functions that use **kernel functions**
- Optimization rather than greedy search
- Statistical learning
 - The search of a prediction function is modeled as a **function estimation** problem

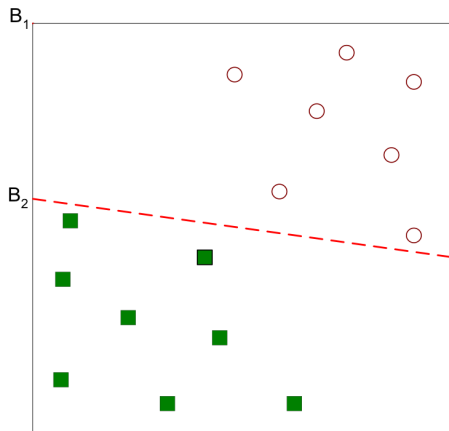
The Maximum Margin hyperplane



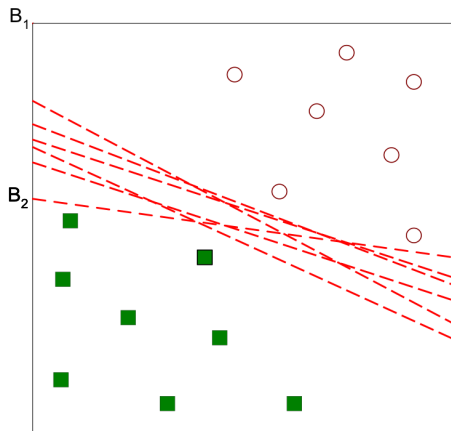
The Maximum Margin hyperplane



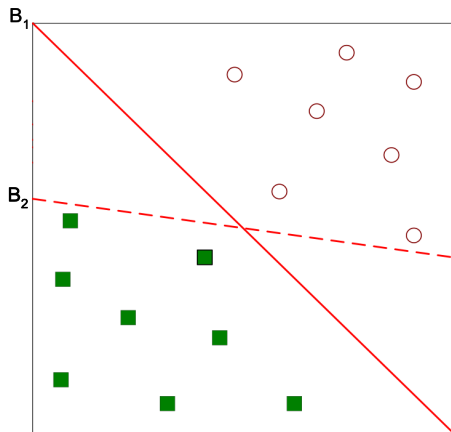
The Maximum Margin hyperplane



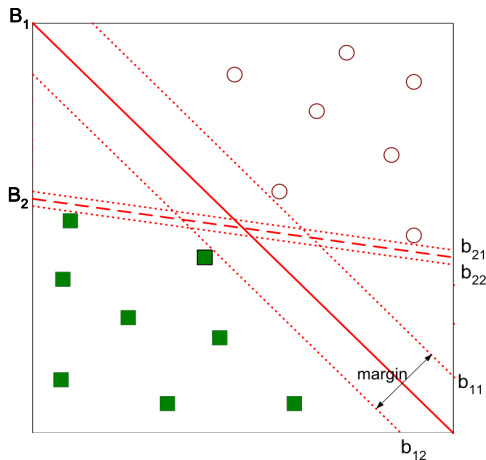
The Maximum Margin hyperplane



The Maximum Margin hyperplane

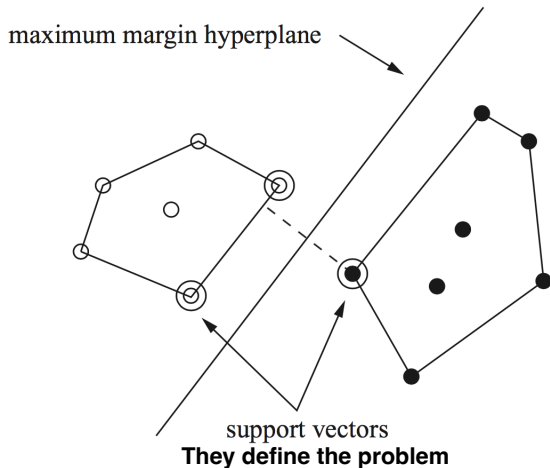


The Maximum Margin hyperplane



Maximum Margin Hyperplane I

- The linear perceptron accepts **any** hyperplane able to separate the classes of the training examples
 - It is conceivable that some hyperplanes are better than others for the classification of **new** items
- The **maximum margin hyperplane** gives the greatest separation between the classes



Maximum Margin Hyperplane II

- The *convex hull* of a set of points is the tightest enclosing convex polygon
 - if the dataset is linearly separable the convex hulls of the classes do not intersect
- The maximum margin hyperplane is as far as possible from both the hulls
 - it is the perpendicular bisector of the shortest line connecting the hulls
- In general a subset of the points is sufficient to define the hull
 - those are the **support vectors** (circled in the figure of the previous page)
- Support vectors are the elements of the training set which **would change**

Maximum Margin Hyperplane III

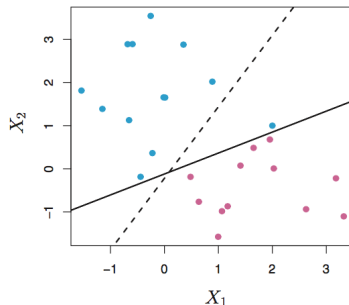
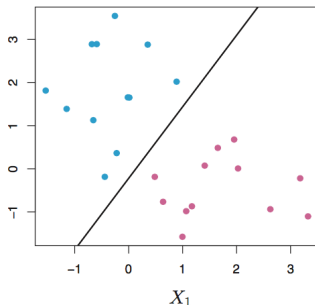
- Finding the support vectors and the maximum margin hyperplane belongs to the well known class of *constrained quadratic optimization* problems

$$\begin{aligned} & \max_{w_0, w_1, \dots, w_D} M \\ & \text{subject to } \sum_{j=1}^D w_j^2 = 1 \\ & c_i(w_0 + w_1 x_{i1} + \dots + w_D x_{iD}) > M, \forall i = 1, \dots, N \end{aligned}$$

where the class of example i is either -1 or 1 and M is the **margin**

Soft margin

- It is quite common that a separating hyperplane does not exist
 - see figure in slide 23 in the perceptron section
- $\Rightarrow$ find an hyperplane which **almost** separate the classes
- $\Rightarrow$ disregard examples which generate a very narrow margin



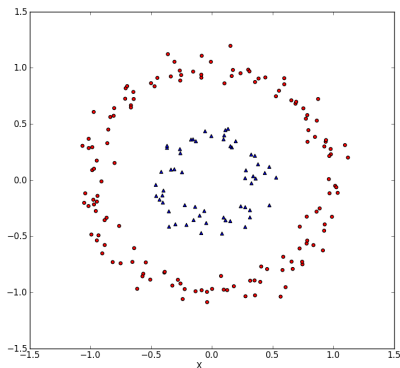
Soft margin Support Vector classifier

- Greater robustness to individual observations
- Better classification of **most** of the training observations
- Obtained by adding a constraint to the optimization problem expressed by a single numeric parameter, usually called C in the literature
 - C , the penalty parameter of the error term, controls the amount of overfitting
 - C tuning is critical

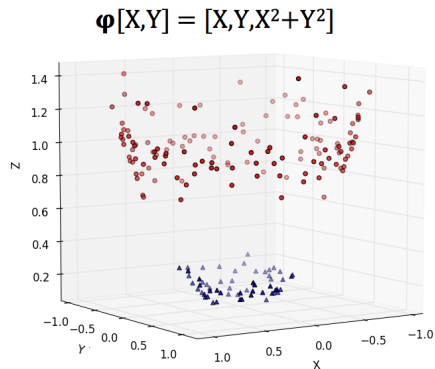
Non-linear class boundaries I

- The support vector method avoids the problem of overfitting
- The nonlinearity of boundaries can be overcome with a **non-linear mapping**
 - the data are mapped into a new space, usually called *feature space*, such that a linear boundary in the feature space can correspond to a non-linear boundary in the original space
 - the feature space can have a number of dimensions higher than the original one

Non-linear class boundaries II

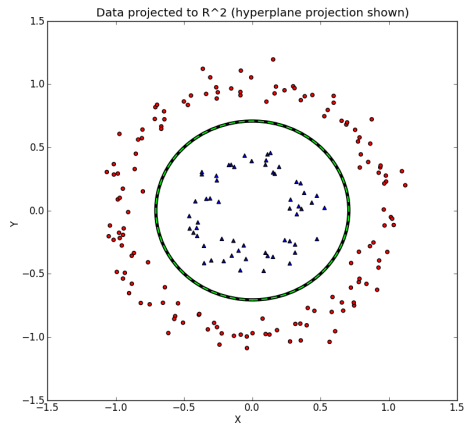
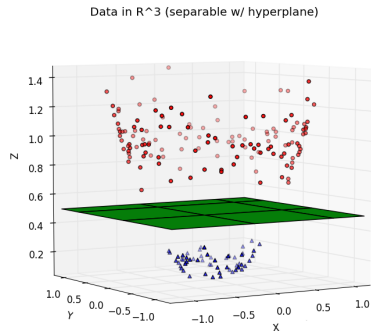


Input space



Feature space

Non-linear class boundaries III



Now linearly separable

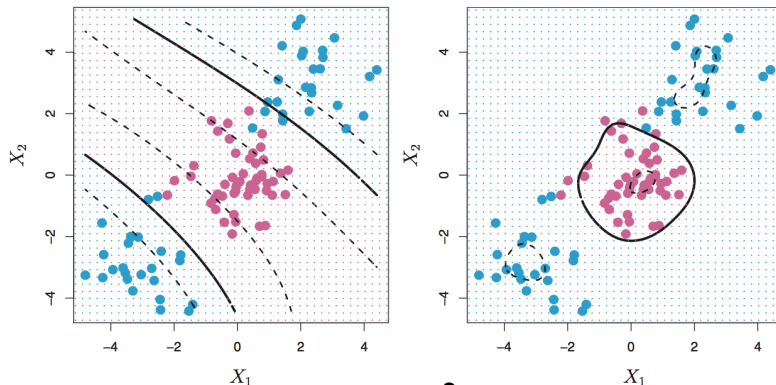
The *kernel trick*

- The separating hyperplane computation requires a series of **dot product** computations among the training data vectors
- Defining the mapping on the basis of a particular family of functions, called **kernel functions**, or simply **kernels**, the mapping does not need to be explicitly computed, and the computation is done in the input space
- This avoids an increase in the complexity
- Some kernel functions:

linear	$\langle x, x' \rangle$
polynomial	$(\gamma \langle x, x' \rangle + r)^{dg}$
rbf	$\exp(-\gamma \ x - x'\ ^2)$
sigmoid	$\tanh(\langle x, x' \rangle + r)$

- γ, dg, r are parameters specified in the documentation of the learning tools
- **Rule of thumb: start with the simpler and then try with the more complex if necessary**

Examples of decision boundaries with kernels



Decision boundaries for polynomial kernel of degree ² (left) and radial based kernel (RBF, right)
In this case they seem to be both appropriate, but the generalization capabilities change,
depending on where new data could be expected

SVM Complexity

- The time complexity is mainly influenced by the efficiency of the optimization library
- The popular libSVM library scales from $\mathcal{O}(D * N^2)$ to $\mathcal{O}(D * N^3)$, depending on the effectiveness of data caching in the library, which is **data dependent**
 - in case of sparse data it is reduced

Final remarks on SVM

- Learning is in general slower than simpler methods, such as decision trees
- Tuning is necessary to set the parameters (not discussed here)
- The results can be very accurate, because subtle and complex decision boundaries can be obtained
- Explicitly based on a theoretical model of learning
- Are not affected by local minima
- Do not suffer from the curse of dimensionality: do not use any notion of **distance**
- SVMs do not directly provide probability estimates, these can be calculated using rather expensive estimation techniques (see `scikit-learn` documentation in References below)
 - nevertheless, SVM can produce a **confidence score** related to the distance of an example from the separation hyperplane

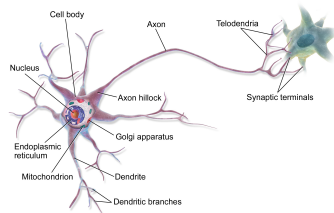
SVM - References

1. See reference [James et al.(2015)James, Witten, Hastie, and Tibshirani] for a more detailed explanation of the maths behind SVM (unfortunately it is not publicly available, ask the teacher for a temporary loan)
2. Jordan, Michael I., and Romain Thibaux. "The Kernel Trick." Lecture Notes. 2004. Web. 5 Jan. 2013. <https://people.eecs.berkeley.edu/~jordan/courses/281B-spring04/lectures/lec3.pdf>
3. Berwick, Robert. "An Idiot's Guide to Support Vector Machines (SVMs)". Lecture Slides. 2003. Web. 5 Jan. 2013. <http://www.cs.ucf.edu/courses/cap6412/fall2009/papers/Berwick2003.pdf>
4. "Scikit-learn: sklearn.svm.SVC Documentation", Pedregosa et al. <http://scikit-learn.org/dev/modules/generated/sklearn.svm.SVC.html>
5. Rifkin, Ryan. "Multiclass Classification". Lecture Slides. February 2008. Web. 6 Jan. 2013. <http://www.mit.edu/9.520/spring09/Classes/multiclass.pdf>
6. Hofmann, Martin. "Support Vector Machines – Kernels and the Kernel Trick". Notes. 26 June 2006. Web. 7 Jan. 2013. http://www.cogsys.wiai.uni-bamberg.de/teaching/ss06/hs_svm/slides/SVM_Seminarbericht_Hofmann.pdf

1	Statistical modeling – Naive Bayes Classifier	2
2	Linear Classification with the Perceptron	17
3	Support Vector Machines	24
4	Neural Networks	42
	• Multi-layer perceptron	44
	• Training the Neural Network	50
	• Computing weight corrections	52
	• Training algorithm	56
	• Example	63
5	Instance Learning - K Nearest Neighbours Classifier	66
6	Loss function	68
7	From a binary classifier to multi-class classification	70
8	Ensemble methods	75
9	Forest of randomised trees	85
10	Boosting	89

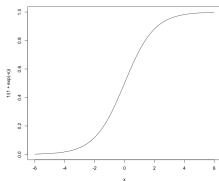
Neural Networks

- Arrange many perceptron-like elements in a hierarchical structure
 - Another way to overcome the limit of linear decision boundary
- Inspired to the complex interconnections of neurons in animal brains
- A neuron is a signal processor with **threshold**
- Signal transmission from one neuron to another is **weighted**
 - weights change over time, also due to **learning**

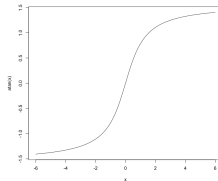


Multi-layer perceptron

- The signals transmitted are modeled as real numbers
- The threshold of the biological system is modeled as a mathematic function
 - continuous and differentiable, superiorly and inferiorly limited
 - the derivative can be expressed in terms of the function itself
 - this simplifies the mathematics
- Several functions available



Sigmoid

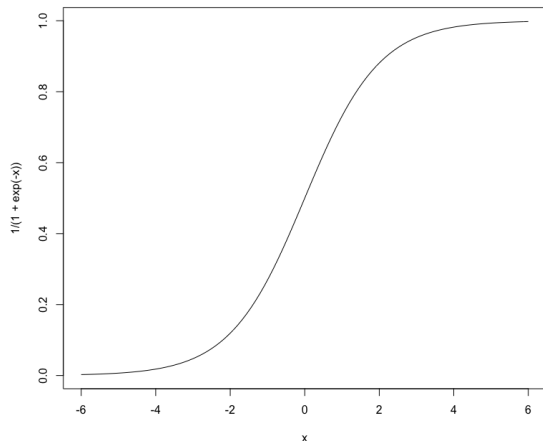


Arctangent

Sigmoid

- Also called *squashing function*
- Maps reals into $]0, 1[$
- Is continuous, differentiable, non-linear

$$\frac{1}{1 + e^{-x}}$$



Importance of non-linearity

- Results with the linear perceptron were non satisfactory because of linearity
 - in addition to the problem of separability
- In a linear system $f(x_1 + x_2) = f(x_1) + f(x_2)$
 - if x_2 is generated by noise, it is completely transferred to the output
- In a non-linear system, in general, $f(x_1 + x_2) \neq f(x_1) + f(x_2)$
- The shape of the function can influence the learning speed

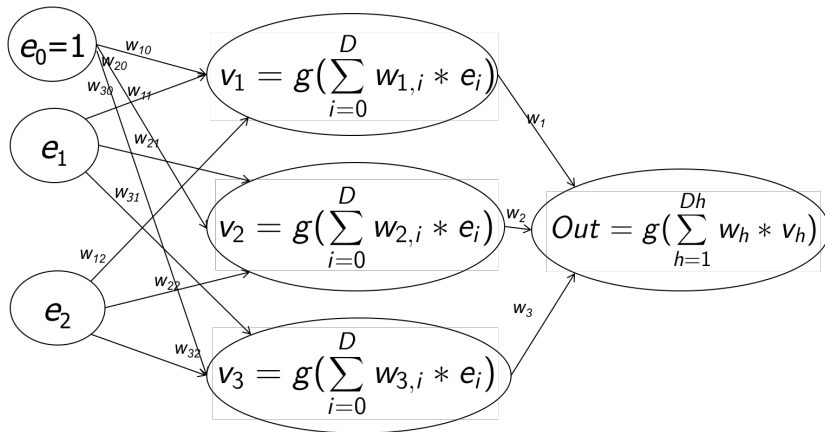
Non-linearity is useful in order to give less importance to noise

Feed-forward multi-layered network

- Inputs feed an **input layer**
 - one input node for each dimension in the training set
- Input layer feeds (with weights) a **hidden layer**
- Hidden layer feeds (with weights) an **output layer**
 - the number of nodes in the hidden layer are a parameter of the network
 - the number of nodes in the output layer is related to number of different classes in the domain
 - one node if there are two classes
 - one node per class in the other cases⁴

⁴ As an alternative, it is possible to adopt the OVO or OVA architecture, see page 71

Feed-forward multi-layered network – Example

 $D_{in} = D = 2$
 $D_h = 3$
 $D_{Out} = 1$ (binary)


1

Details

- $g(.)$ is the transfer function of the node, e.g. the sigmoid
- The unitary input x_0 is added for dealing with the bias, as in the case of the linear perceptron
- The weights are for each edge connecting two nodes
- **Feed-forward** defines which oriented edges are present
 - edges connect only a node in a layer to a node in the following layer
 - input to hidden and hidden to output
 - each node of one layer is connected to all the nodes of the following layer
- In this way the signal flows from the input to the output, without loops

Training the Neural Network I

set all weights to random values

while termination condition is not satisfied **do**

for each training instance x **do**

 feed the network with x and compute the output $nn(x)$

 compute the weight corrections for $nn(x) - x_{Out}$

 propagate back the weight corrections


Ground truth

Training the Neural Network II

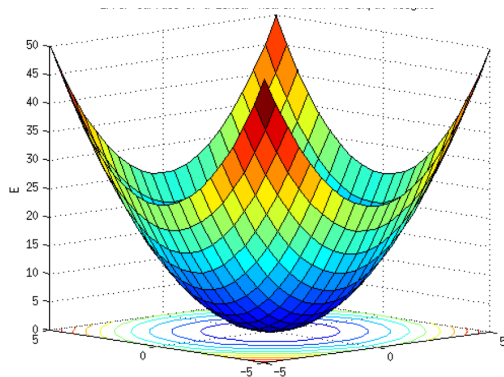
- In analogy with learning in the animal domain, the examples must repeatedly feed the network
- The weights **encode** the knowledge given by the supervised examples
- The encoding is not easily understandable: it looks like a structured set of real numbers
- Convergence is not guaranteed
- Important issues:
 - computing the weight corrections
 - preparation of the training examples
 - standardize the attributes to have zero mean and unit variance
 - termination condition

Computing the error

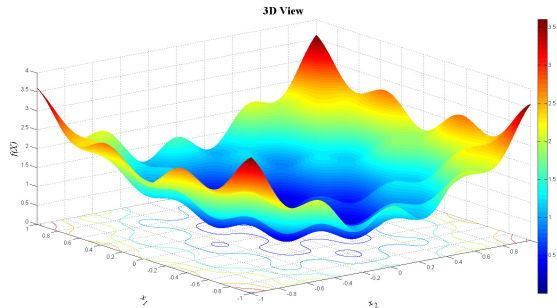
- Let $\mathbf{x}$ and y be the input vector and the desired output of a node, respectively
- Let $\mathbf{w}$ be the input weight vector of a node
- The error is:

$$E(\mathbf{w}) = \frac{1}{2}(y - \text{Transfer}(\mathbf{w}, \mathbf{x}))^2$$

Error functions



Convex error function



Non convex error function

Computing the gradient I

- Move towards a (local) minimum of the error
 - follow the **gradient**
 - compute the partial derivatives of the error as a function of the weights

$$\text{sgm}(x) = \frac{1}{1 + e^{-x}}$$
$$\frac{d}{dx}\text{sgm}(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = (1 - \text{sgm}(x))\text{sgm}(x)$$

Computing the gradient II

- The weight is changed subtracting the partial derivative multiplied by a **learning rate** constant
 - the learning rate influences the convergence speed and can be adjusted as a tradeoff between speed and precision
- The subtraction moves towards smaller errors
- The derivatives of the input weights of the nodes of a layer can be computed if the derivatives for the following layer are known
- *The actual derivatives are omitted here*

$$w_{ij} \leftarrow w_{ij} - \lambda \frac{\partial E(\mathbf{w})}{\partial w_{ij}}$$

Training algorithm – revised

set all weights to random values

while termination condition is not satisfied **do**

for each training instance x **do**

 1 – feed the network with x and compute the output $nn(x)$

 2 – compute error prediction at output layer $nn(x) - x_{Out}$

 3 – compute derivatives and weight corrections for output layer

 4 – compute derivatives and weight corrections for hidden layer

Steps 1 and 2 are *forward*, steps 3 and 4 are *backward*

Learning modes

- Stochastic** – each forward propagation is immediately followed by a weight update (as in the algorithm of previous slide)
- introduces some *noise* in the gradient descent process, since the gradient is computed from a single data point
 - reduces the chance of getting stucked in a local minimum
 - good for *online* learning



- Batch** – many propagations occur before updating the weights, accumulating errors over the samples within a batch
- generally yields faster and stable descent towards the *local* minimum, since the update is performed in the direction of the average error

Repetitions

- A learning round over all the samples of the network is called **epoch**
- In general, after each epoch the network classification capability will be improved
- Several epochs will be necessary
- After each epoch the starting weights will be different

Design choices

- The structure of input and output layers is determined by the domain (the training set)
- The number of nodes in the hidden layer can be changed
- The learning rate can be changed in different epochs
 - in the beginning a higher learning rate can push faster towards the desired direction
 - in later epochs a lower learning rate can push more precisely towards a minimum

Stop criteria

- All the weight updates in the epoch have been small
- The classification error rate goes below a predefined target
- A timeout condition is reached

Risks

- **Local minima** are possible, as usual in gradient tracking methods
- **Overfitting** is possible, if the network is too complex w.r.t. the complexity of the decision problem

Regularization

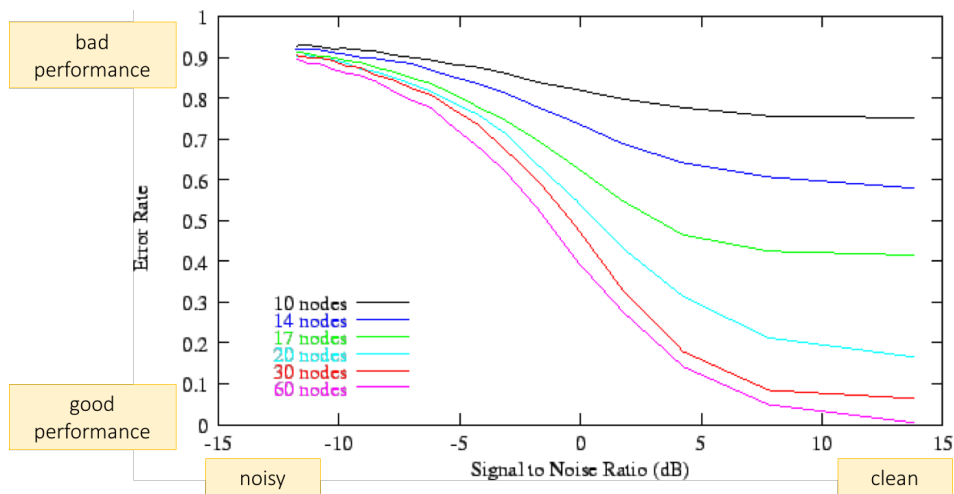
Like reducing the tree-depth

- Technique used in many machine learning functions to improve the generalisation capabilities of a model
- Modify the performance function, which is normally chosen to be the sum of squares of the network errors on the training set
- In essence:
 - improvement of performance is obtained by reducing a **loss function** (i.e. the sum of squared errors)
 - regularisation corrects the loss function in order to **smooth** the fitting to the data
 - the amount of regularisation must be tuned

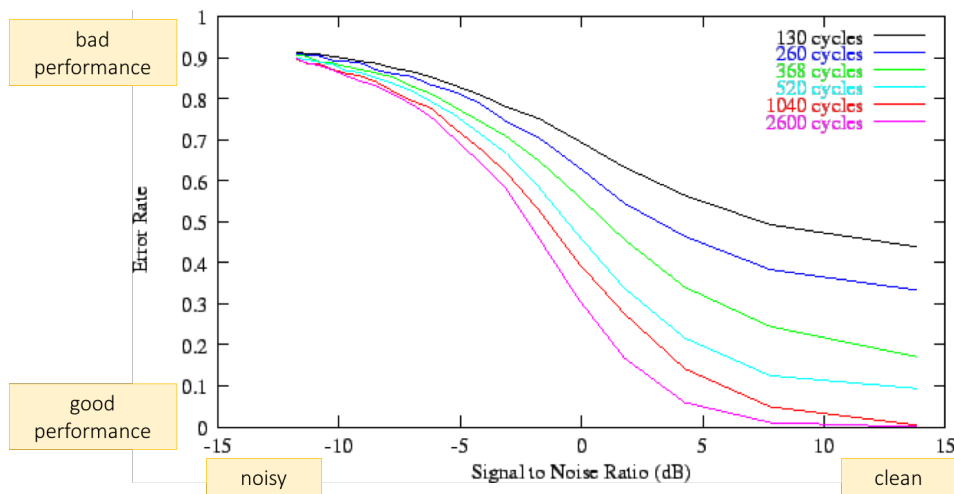
Example

- Recognition of characters with noise
- The images are 7x5 arrays of floating points
 - before noise -1 for black, 1 for white
- 35 input nodes
- up to 60 hidden nodes
- 26 output nodes
 - only one at a time should give a value near 1, all the others should be near 0
 - a good alternative could be a 5 bit coding

Error rate — Varying signal/noise and hidden nodes



Error rate – Varying signal/noise and number of epochs



1	Statistical modeling – Naive Bayes Classifier	2
2	Linear Classification with the Perceptron	17
3	Support Vector Machines	24
4	Neural Networks	42
5	Instance Learning - K Nearest Neighbours Classifier	66
6	Loss function	68
7	From a binary classifier to multi-class classification	70
8	Ensemble methods	75
9	Forest of randomised trees	85
10	Boosting	89
11	Summary	95

K Nearest Neighbours Classifier

- keeps all the training data
 - i.e. the *model* is the entire training set
- future predictions by computing the similarity between the new sample and each training instance
 - can use any similarity function
- picks the K entries in the database which are closest to the new data point
- majority vote
- main parameters
 - the number of neighbours to check
 - the *metric* used to compute the distances
 - the *Mahalanobis* distance has good performance

We will use:

- Euclidean
- Mahalanobis
- Manhattan

1	Statistical modeling – Naive Bayes Classifier	2
2	Linear Classification with the Perceptron	17
3	Support Vector Machines	24
4	Neural Networks	42
5	Instance Learning - K Nearest Neighbours Classifier	66
6	Loss function	68
7	From a binary classifier to multi-class classification	70
8	Ensemble methods	75
9	Forest of randomised trees	85
10	Boosting	89
11	Summary	95

What is a loss function

We used up to now **performance scores** and we always tried to maximise them

*The function we want to minimise or maximise is called the objective function or criterion. When we are minimising it, we may also call it the **cost function**, **loss function**, or **error function***

Ian Goodfellow, Yoshua Bengio, Aaron Courville

Many machine learning algorithms, in particular those based on **gradient descent**, are designed to minimise the loss; multi-layer perceptron is one of them.

1	Statistical modeling – Naive Bayes Classifier	2
2	Linear Classification with the Perceptron	17
3	Support Vector Machines	24
4	Neural Networks	42
5	Instance Learning - K Nearest Neighbours Classifier	66
6	Loss function	68
7	From a binary classifier to multi-class classification	70
8	Ensemble methods	75
9	Forest of randomised trees	85
10	Boosting	89
11	Summary	95

From a binary classifier to multi-class classification⁵

- Several classifiers (e.g. SVM and linear perceptron), generate a **binary** classification model
- two ways to deal with multi-class classification
 - transform the training algorithm and the model
 - sometimes at the expenses of an increased size of the problem
 - use a set of binary classifiers and combine the results
 - at the expenses of an increased number of problems to solve
 - **one-vs-one** and **one-vs-rest** strategies

5 [James et al.(2015)James, Witten, Hastie, and Tibshirani] chapter 9

One-vs-one strategy (OVO)

If we have the classes A,B,C we will generate 3 classifiers: AB, AC, BC

Then when i want to classify an element X, i will submit it to all of those 3 classifiers and maybe i will obtain different options. I will choose the majority



- consider all the possible pairs of classes and generate a binary classifier for each pair
 - $C * (C - 1) / 2$ pairs
- each binary problem considers only the examples of the two selected classes
- at prediction time apply a **voting** scheme
 - an unseen example is submitted to the $C * (C - 1) / 2$ binary classifiers, each winning class receives a +1
 - the class with the highest number of +1 wins

One-vs-rest strategy (OVR)

Assuming the example as before: classes A,B,C
i will consider those comparisons:

A vs (BC) --> +

B vs (AC) --> +

C vs (AB) --> -

- consider C binary problems where class c is a positive example, and all the others are negatives
- build C binary classifiers
- at prediction time apply a **voting** scheme
 - an unseen example is submitted to the C binary classifiers, obtaining a confidence score
 - the confidences are combined and the class with the highest global score is chosen

OVO vs OVR

- OVO requires solving a higher number of problems, even if they are of smaller size
- OVR tends to be intrinsically unbalanced
 - if the classes are evenly distributed in the examples, each classifier has a proportion positive to negative 1 to $C - 1$

1	Statistical modeling – Naive Bayes Classifier	2
2	Linear Classification with the Perceptron	17
3	Support Vector Machines	24
4	Neural Networks	42
5	Instance Learning - K Nearest Neighbours Classifier	66
6	Loss function	68
7	From a binary classifier to multi-class classification	70
8	Ensemble methods	75
●	● Methods for ensemble classifiers	80
9	Forest of randomised trees	85
10	Boosting	89
11	Summary	95

Ensemble methods

aka Classifier combination

- train a set of **base classifiers**
- the final prediction is obtained taking the votes of the base classifiers
- ensemble methods tend to perform better than a single classifier

Why should this be better?

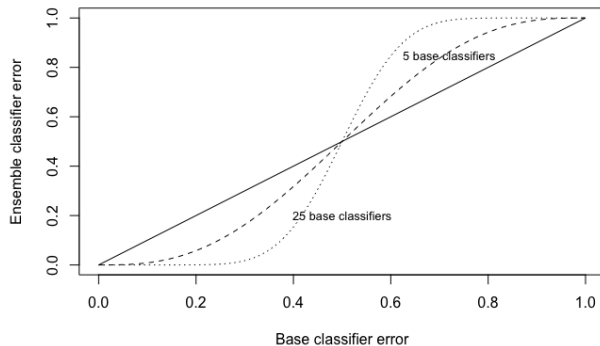
A hypothetical, extreme case

- let us consider 25 binary classifiers, each of which has **error rate** $\epsilon = 0.35$
- the ensemble classifier output is the majority of the predictions (13 or more)
- if the base classifiers are equal, the ensemble error rate is still ϵ
- if the base classifiers have all error rate ϵ , but they are independent, i.e. their errors are uncorrelated, then the ensemble will be wrong only when the majority of the base classifier is wrong

$$\epsilon_{ensemble} = \sum_{i=13}^{25} \binom{25}{i} \epsilon^i (1 - \epsilon)^{25-i} = 0.06$$

Error in $n \geq 13$ examples

Ensemble error – a hypothetical extreme case



Comparison between error of the base classifiers and error of the ensemble classifier

Rationale for ensemble method

Ensemble methods are useful if:

1. the base classifiers are independent
2. the performance of the base classifier is better than random choice

Methods for ensemble classifiers I

By manipulating the training set -

Data are resampled according to some sampling strategy

Bagging repeatedly samples with replacement according to a uniform probability distribution

Boosting iteratively changes the distribution of training examples so that the base classifier focus on examples which are hard to classify

Adaboost the importance of each base classifier depends on its error rate

Methods for ensemble classifiers II

By manipulating the input features -

Subset of input features can be chosen either random or according to suggestions from domain experts

Random forest uses decision trees as base classifiers

- frequently produces very good results

Methods for ensemble classifiers III

By manipulating the class labels -

Useful when the number of classes is high

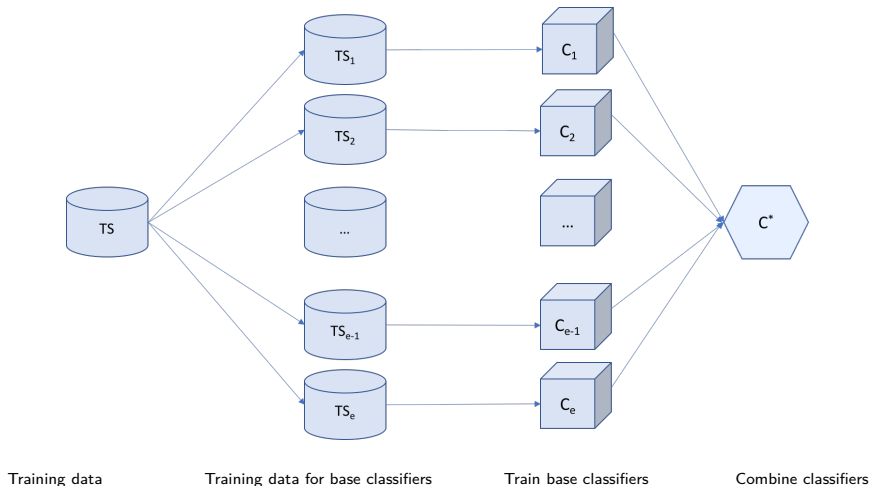
For each base classifier, randomly partition class labels into two subsets, say A_1, A_2 and re-label the dataset

Train binary classifiers with respect to the two classes

At testing time, when a subset is selected all the classes it includes receive a vote

The class with the top score will win

General schema for ensemble methods



1	Statistical modeling – Naive Bayes Classifier	2
2	Linear Classification with the Perceptron	17
3	Support Vector Machines	24
4	Neural Networks	42
5	Instance Learning - K Nearest Neighbours Classifier	66
6	Loss function	68
7	From a binary classifier to multi-class classification	70
8	Ensemble methods	75
9	Forest of randomised trees	85
10	Boosting	89
11	Summary	95

Forest of randomised trees

- perturb-and-combine techniques specifically designed for trees
- a diverse set of classifiers is created by introducing randomness in the classifier construction
 - the classifiers must be independent
 - each tree in the ensemble is built from a sample drawn with replacement (i.e., a bootstrap sample) from the training set
 - furthermore, when splitting each node during the construction of a tree, the best split is found either from all input features or a random subset of size `max_features`
- the prediction of the ensemble is given as the combination of the predictions of the individual classifiers (e.g. voting)

Bias-vs-Variance tradeoff⁶

- **Bias** is the simplifying assumptions made by the model to make the target function easier to approximate
- **Variance** is the amount that the estimate of the target function will change, given different training data
- Bias-variance trade-off is the sweet spot where our machine model performs between the errors introduced by the bias and the variance.

*If Bias vs Variance was the act of reading, it could be like
Skimming a Text vs Memorizing a Text*

6 <https://www.kdnuggets.com/2020/09/understanding-bias-variance-trade-off-3-minutes.html>

Random Forest

the purpose of the two sources of randomness is to decrease the variance of the forest estimator

- individual decision trees typically exhibit high variance and tend to overfit
- the injected randomness in forests yield decision trees with somewhat decoupled prediction errors
- by taking an average of those predictions, some errors can cancel out
- random forests achieve a reduced variance by combining diverse trees, sometimes at the cost of a slight increase in bias.
- In practice the variance reduction is often significant hence yielding an overall better model.

1	Statistical modeling – Naive Bayes Classifier	2
2	Linear Classification with the Perceptron	17
3	Support Vector Machines	24
4	Neural Networks	42
5	Instance Learning - K Nearest Neighbours Classifier	66
6	Loss function	68
7	From a binary classifier to multi-class classification	70
8	Ensemble methods	75
9	Forest of randomised trees	85
10	Boosting	89
11	Summary	95

Ensemble learning with *boosting*

- a weight is associated to each training instance
- different classifiers are trained with those weights
- the weights are modified iteratively according to classifier performance

AdaBoost I

- fit a sequence of weak learners on repeatedly modified versions of the data
- the predictions from all of them are then combined through a weighted majority vote (or sum) to produce the final prediction
- the data modifications at each so-called boosting iteration consist of applying and modifying weights to each of the training samples

AdaBoost II

- initially, the weights $w_1, w_2, \dots, w_N$ are all set to $1/N$, so that the first step simply trains a weak learner on the original data
- for each successive iteration, the sample weights are individually modified and the learning algorithm is reapplied to the *reweighted* data
 - at a given iteration, the training examples that were incorrectly predicted by the boosted model induced at the previous step have their weights increased, whereas the weights are decreased for those that were predicted correctly

AdaBoost III

We obtain that after a certain number of iterations the classifiers are more focused on the misclassified examples

- as iterations proceed, examples that are difficult to predict receive ever-increasing influence; each subsequent weak learner is thereby forced to concentrate on the examples that are missed by the previous ones in the sequence

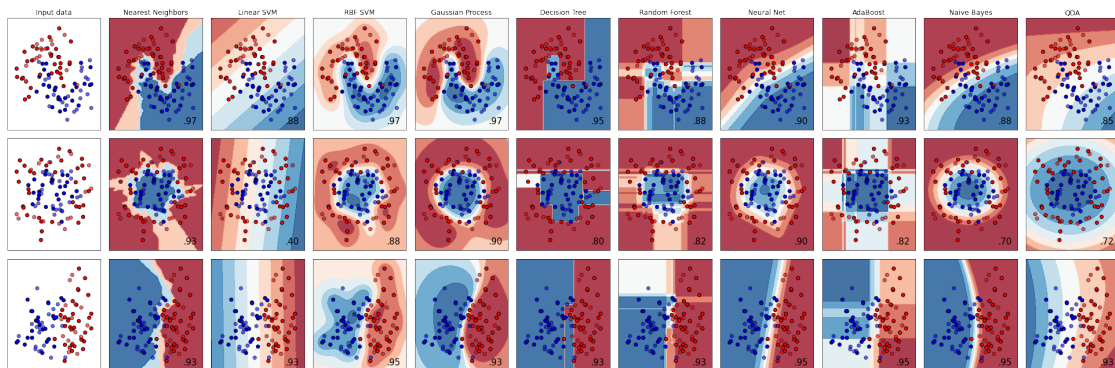
Other algorithms

- XGBoost** (Extreme Gradient Boosting) stands for eXtreme Gradient Boosting, is a popular and powerful machine learning algorithm used for both classification and regression tasks. It is an implementation of gradient boosted decision trees designed for speed and performance
- LightGBM** gradient boosting framework developed by Microsoft that is designed for distributed and efficient training. It is known for its high performance and supports large datasets.
- CatBoost** is a gradient boosting library developed by Yandex that is designed to handle categorical features efficiently. It is known for its ease of use and ability to perform well with default hyperparameters.
- Gradient Boosting Machines** general term that refers to the class of algorithms that build a model in a stage-wise fashion by optimising a differentiable loss function.
 - H2O.ai** is an open-source software for data analysis that includes an implementation of gradient boosting. It is designed for distributed computing and is known for its speed and scalability.
- TensorFlow Decision Forests** is a library developed by Google that provides an implementation of decision forests, which includes both Random Forests and Gradient Boosted Decision Trees (GBDT).
- Sklearn Gradient Boosting** or Gradient Boosted Decision Trees (GBDT) is a generalization of boosting to arbitrary differentiable loss functions, see the seminal work of [Friedman2001]. GBDT is an excellent model for both regression and classification, in particular for tabular data.

1	Statistical modeling – Naive Bayes Classifier	2
2	Linear Classification with the Perceptron	17
3	Support Vector Machines	24
4	Neural Networks	42
5	Instance Learning - K Nearest Neighbours Classifier	66
6	Loss function	68
7	From a binary classifier to multi-class classification	70
8	Ensemble methods	75
9	Forest of randomised trees	85
10	Boosting	89
11	Summary	95

Classifiers comparison

The scales of red and blue are the decision margins when transitioning from a class to the other



Bibliography

- ▶ Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani.
An Introduction to Statistical Learning.
Springer, 2015.
- ▶ Ian H. Witten, Eibe Frank, and Mark Hall.
Data Mining – Practical Machine Learning Tools and Techniques.
Morgan Kaufman, 2011.
ISBN 978-0-12-374856-0.

1) Euclidean distance
Minkowski distance is a generalization of euclidean (p=2) and manhattan (p=1)

1) MINKOWSKI

$$d(x, y, p) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p} \longrightarrow \begin{array}{l} p=1 \Rightarrow \text{Manhattan distance} \\ p=2 \Rightarrow \text{Euclidean distance} \end{array}$$

2) MALANOBIS

$$D_M(x) = \sqrt{(x - \mu)^T \underline{S^{-1}} (x - \mu)}$$

Covariance Matrix $\longrightarrow$ Correlation Matrix Normalized